ORIEL 0.7.3

Modules and Standard Library

Development, implementation status and local verification report

Release objective
Provide dependable multi-file projects, deterministic module resolution and the first bundled ORIEL standard library while preserving the shared multi-IDE compiler-services architecture.

Prepared for the ORIEL open-source repository
July 2026

1. Release Summary

ORIEL 0.7.3 advances the project from static type-system completion to dependable modules and standard-library foundations. The release adds a structured module graph API, deterministic dependency ordering, dotted and relative imports, circular dependency diagnostics, alias and public-import metadata, safe path containment, and bundled standard-library source modules.

Capability	Result
Version metadata	Updated to 0.7.3
Automated tests	47 passed
Python source compilation	Passed
CLI version command	Oriel 0.7.3
Module example check	Passed
Module example execution	Outputs 5 and 36
Wheel build	Not run locally: Python build package unavailable

2. Implemented Module Capabilities

• Deterministic dependency-first module ordering.

• Dotted imports such as `use tools.math`.

• Relative source imports such as `use "helpers.orl"`.

• Alias metadata such as `use tools.math as maths`.

• Public import metadata for future re-export/linker support.

• Circular dependency detection with readable cycle paths.

• Duplicate alias diagnostics.

• Project-root containment that rejects path traversal outside the project.

• Cross-platform path normalization and UTF-8 source loading.

3. Standard Library Modules

Module	Initial purpose
oriel.core	Identity and basic value helpers
oriel.text	Text length and empty checks
oriel.math	Square and clamp helpers
oriel.collections	Collection size and empty checks
oriel.files	Text file load/save wrappers
oriel.json	JSON encode/decode wrappers
oriel.time	Clock/time wrapper
oriel.config	Environment configuration wrapper
oriel.logging	Info, warning and error logging helpers

Module	Initial purpose
oriel.testing	Basic assertion helpers

4. Multi-IDE Architecture Alignment

The module implementation remains centralized in the ORIEL compiler-services layer. IDE plugins must not implement their own import resolver. VS Code, JetBrains, Android Studio, Visual Studio, Eclipse, Neovim, Vim, Emacs, Helix, Zed, Fleet, Kate and Sublime Text should consume the same module graph and diagnostics through the shared Language Server Protocol service.

5. Known Limitation

The current bootstrap interpreter executes a flattened module graph. Import aliases and public-import markers are parsed and exposed through the module API, but namespace-qualified symbol access and strict private-symbol enforcement are not yet complete. Those capabilities require a dedicated AST module linker and the future ORIEL intermediate representation. The repository documentation marks this accurately as partially implemented rather than complete.

6. Next Release Direction

ORIEL 0.7.4 - Package Manager should add a stable manifest and lock-file model, semantic version resolution, transitive dependencies, local/Git/registry sources, checksums, offline cache, publishing workflow and isolated CI registry tests. The module resolver and package manager should share one canonical package/module location model.

7. Release Gate

The release is ready for source review and CI integration. A genuine tagged release should be created only after Windows, Ubuntu and macOS CI pass, the wheel is built, standard-library package data is verified from an isolated wheel installation, and release artifacts are generated from the tagged commit.